

# 作业 9

## ——汉明距离<sup>1</sup>最近邻查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

### 1. 题目

#### 描述

有一个包含  $n$  个长度为64的01串的集合  $A$  以及  $q$  个询问，每次询问会给出一个长度为64的01串  $x$ ，你需要找到在  $A$  中， $x$  在汉明距离下的最近邻的 5-近似，即，假设在  $A$  中与  $x$  的汉明距离最短的是  $y$ ，你需要找到  $z \in A$ ，使得  $\text{dist}(x, z) \leq 5 * \text{dist}(x, y)$ 。这里  $\text{dist}(x, y)$  表示  $x$  与  $y$  的汉明距离。注意，可能存在多个 5-近似的最近邻，你只需要输出其中任意一个即可。

#### 输入

第一行两个正整数  $n, q \leq 100000$ 。

接下来  $n$  行，第  $i$  行是一个不超过  $2^{64}$  的正整数  $a_i$ ，它的二进制表示一个 01 串（若长度不足 64 位，则补前导零）。

接下来  $q$  行，第  $i$  行是一个不超过  $2^{64}$  的正整数  $x_i$ ，表示一个询问。

#### 输出

你需要输出  $q$  行，第  $i$  行是一个在 1 到  $n$  之间的正整数  $t_i$ ，表示  $a_{t_i}$  是  $x_i$  在汉明距离下的最近邻的 5-近似。

#### 样例输入

```
5 5
679
793
129
91
406
613
98
534
752
595
```

#### 样例输出

```
1
4
5
1
4
```

<sup>1</sup>汉明距离是以理查德·卫斯里·汉明命名的概念，以  $d(x, y)$  表示两个字  $x, y$  之间的汉明距离，指两个等长字符串对应位置不同字符的数量。

## 2. 思路

由于数据数目很大，我们不能一个一个去数，否则必然超时。这道题需要介绍的算法叫做 LSH (Locality-Sensitive Hashing)，它的核心思想是通过构造一些哈希函数，使得相似的输入在哈希空间中也相似，从而可以快速地找到近似最近邻。

具体来说，我们的操作是，将每个 64 位的 001 串进行重新排列，然后检查前面  $k$  位是否相同，如果相同则认为它们是近似的。假设我们检查了前 10 个数字，他们都相同，那么最多这两个数字串有 54 个数字是不对应相等的。而这只是一个重排之后的结论。我们多用一些重排的方法，就可以控制住近似的程度。

接下来，我们详细来说说这个题目本身。首先，我们得到的输入是十进制的整数，我们需要自己把它转化成二进制下的 01 串。其次，我们得到的 query 也是整数，也需要进行这样的转换，因此，我们为了简化问题，重复使用代码，写出这个转换的过程：

```
void Trans(ull x, vector<int>& binaryString) {
    for(int i = 0; i < 64; i++){
        if(x != 0){
            binaryString[i] = x % 2;
            x /= 2;
        }
        else
            binaryString[i] = 0;
    }
}
```

于是，问题的关键就转化到了如何实现将一个串进行重排。我们可以通过一个随机数生成器来生成一个长度为 64 的随机排列，然后按照这个排列来重排每个串。具体来说，我们可以使用 C++ 中的 `std::shuffle` 函数来实现这个功能。`shuffle` 函数可以随机打乱一个范围内的元素，因此我们可以创建一个包含 0 到 63 的数组，然后使用 `shuffle` 来打乱它，得到一个随机的排列。再将原来第  $i$  位的数字放到新 `shuffle` 排列得到的位置上，就完成了重排。

```
ull reorder(const vector<int>& binaryString, const vector<int>& p) {
    ull result = 0;
    ull power_of_two = 1;

    for (int newpos = 0; newpos < 64; newpos++) {
        int original_position = p[newpos];
        if (binaryString[original_position] == 1) {
            result += power_of_two;
        }
        power_of_two *= 2;
    }

    return result;
}
```

我们的这个重排函数返回的是重排后的二进制数在 10 进制下的表示。

另外，我们显然还需要一个计算汉明距离的函数：

```
int HamDis(const vector<int>& a, const vector<int>& b) {
    int dis = 0;
    for(int i = 0; i < 64; i++){
        if(a[i] != b[i])
            dis++;
    }
    return dis;
}
```

准备工作就做完啦，我们现在开始写 main 函数。

需要注意的是，重排的方式必须相同才能进行前缀序列的比较，否则完全没有意义。

```
int main(){
    int n, q;
    scanf("%d%d", &n, &q);
    vector<ull> a(n);
    vector<vector<int>> binaryString(n, vector<int>(64));
    for (int i = 0; i < n; i++) {
        scanf("%llu", &a[i]);
        Trans(a[i], binaryString[i]);
    }

    vector<ull> queries(q);
    for (int i = 0; i < q; i++) {
        scanf("%llu", &queries[i]);
    }
    vector<vector<int>> queryBinaryStrings(q, vector<int>(64));
    for (int i = 0; i < q; i++) {
        Trans(queries[i], queryBinaryStrings[i]);
    }

    const int N = 20;
    vector<vector<int>> perm(N, vector<int>(64));
    mt19937 rng(12345);
    for (int k = 0; k < N; k++) {
        for (int i = 0; i < 64; i++) {
            perm[k][i] = i;
        }
        shuffle(perm[k].begin(), perm[k].end(), rng);
    }
}
```

这里没有使用 rand() 函数，而是使用了随机性更强的 std::mt19937，并且设置了一个固定的种子 (12345)，以确保每次运行程序时生成的随机排列都是相同的，这样可以保证结果的一致性。

接下来处理每个询问：

```
const int M = 10;
vector<vector<pair<ull, int>>> tables(N);

for (int k = 0; k < N; k++) {
    tables[k].resize(n);
    for (int i = 0; i < n; i++) {
        ull reordered_val = reorder(binaryString[i], perm[k]);
        tables[k][i] = make_pair(reordered_val, i);
    }
    sort(tables[k].begin(), tables[k].end());
}

vector<int> candidates;
candidates.reserve(N * 2 * M);

for (int qi = 0; qi < q; qi++) {
    candidates.clear();

    for (int k = 0; k < N; k++) {
        ull reordered_query = reorder(queryBinaryStrings[qi], perm[k]);
        int pos = lower_bound(
```

```

        tables[k].begin(),
        tables[k].end(),
        make_pair(reordered_query, -1)
    ) - tables[k].begin();

    int start = max(0, pos - M);
    for (int j = start; j < pos; j++) {
        candidates.push_back(tables[k][j].second);
    }

    int end = min(n, pos + M);
    for (int j = pos; j < end; j++) {
        candidates.push_back(tables[k][j].second);
    }
}

int best_idx = candidates[0];
int best_dist = HamDis(binaryString[best_idx], queryBinaryStrings[qi]);
for (int i = 1; i < candidates.size(); i++) {
    int idx = candidates[i];
    int dist = HamDis(binaryString[idx], queryBinaryStrings[qi]);
    if (dist < best_dist) {
        best_dist = dist;
        best_idx = idx;
    }
}
printf("%d\n", best_idx + 1);
}
return 0;

```

`lower_bound`: C++ 标准库的二分查找函数，返回第一个大于等于目标值的元素的位置。

`make_pair(reordered_query, -1)`: 排序表中的每个元素是 `pair<ull, int>`，比较时先比第一个元素（重排后的值），再比第二个元素（原始下标）。我们用 `-1` 作为第二个元素，因为 `-1` 是最小的整数，这样可以确保：

- 1) 如果排序表中有元素的重排值等于 `reordered_query`，`lower_bound` 会返回第一个这样的元素；
- 2) 如果没有，会返回第一个比它大的元素的位置。

`tables[k].begin()`: 把迭代器转换成整数下标，方便后续操作。

至此，我们成功解答完这道题目，我的咖啡也喝得只剩一半了。

我会把这些题解全部上传到我的 GitHub 上，地址是：

[https://github.com/James-Yao-xj/PKU\\_Program\\_Practice\\_HW](https://github.com/James-Yao-xj/PKU_Program_Practice_HW)

如果你也对算法和编程感兴趣，欢迎来我的 GitHub!